

Here's a **lab report** format for the **2D Geometric Transformations** program you provided:

Title: 2D Geometric Transformations Using OpenGL

Objective:

The objective of this lab is to understand and implement basic 2D geometric transformations on polygons using OpenGL. The transformations include **translation**, **scaling**, **rotation**, **reflection**, and **shearing**. By using these transformations, the program modifies the position, size, and orientation of a polygon and visualizes the effects interactively.

Theory:

In 2D computer graphics, geometric transformations are used to manipulate shapes (polygons) within a coordinate system. Each transformation applies a specific mathematical operation to the vertices of a polygon.

1. **Translation:** Moves a polygon by a given offset along the x and y axes. $[x' = x + t_x, y' = y + t_y]$ where (t_x) and (t_y) are translation factors.
 2. **Scaling:** Changes the size of a polygon by scaling factors along the x and y axes. $[x' = (x - f_x) \cdot s_x + f_x, y' = (y - f_y) \cdot s_y + f_y]$ where (s_x) and (s_y) are the scaling factors, and (f_x) , (f_y) are the fixed points.
 3. **Rotation:** Rotates a polygon about a pivot point by a certain angle. $[x' = (x - p_x) \cdot \cos(\theta) - (y - p_y) \cdot \sin(\theta) + p_x, y' = (x - p_x) \cdot \sin(\theta) + (y - p_y) \cdot \cos(\theta) + p_y]$ where (θ) is the rotation angle, and (p_x) , (p_y) are the coordinates of the pivot point.
 4. **Reflection:** Flips the polygon across an axis (x-axis, y-axis, or origin).
 - Reflection across x-axis: $(y' = -y)$
 - Reflection across y-axis: $(x' = -x)$
 - Reflection across the origin: $(x' = -x, y' = -y)$
 - Reflection across line $(x = y)$: $(x' = y, y' = x)$
 5. **Shearing:** Distorts the polygon by shifting its vertices along the x or y axis.
 - Shearing along x-axis: $(x' = x + sh \cdot y)$
 - Shearing along y-axis: $(y' = y + sh \cdot x)$ where (sh) is the shearing factor.
-

Algorithm:

1. **Input:** The user enters the number of vertices for the polygon and the coordinates of each vertex.
2. **Display:** The polygon is drawn on the screen using OpenGL.
3. **Transformation Selection:** The user selects a transformation (translation, scaling, rotation, reflection, or shearing) from a menu.

4. Apply Transformation:

- The selected transformation is applied to the vertices of the polygon using the respective mathematical formulas.
- The transformed polygon is stored in a new vector and displayed in a different color.

5. **Output:** The transformed polygon is rendered on the screen alongside the original polygon for comparison.

6. **Loop:** The user can repeat the process by selecting different transformations.

Input/Output:

- **Sample Input:**

- Polygon with 3 vertices: ((0, 0), (50, 0), (25, 50))
- Translation by ((20, 10))
- Scaling with factors ((2, 2)) around the fixed point ((0, 0))
- Rotation by 45 degrees about the origin.

- **Sample Output:**

- The original polygon in red and the transformed polygon in green are displayed on the screen. The polygons are rendered side by side for easy visualization of the transformations.
-

Discussion:

Several challenges may arise when implementing 2D geometric transformations as a beginner:

1. **Coordinate System Setup:** Setting up the OpenGL coordinate system correctly is crucial for visualizing the transformations. Misconfiguring the `gluOrtho2D` function may cause polygons to render outside the visible window or appear incorrectly scaled.
2. **Mathematical Errors:** Each transformation has its specific mathematical formulas. Beginners often make mistakes such as incorrect scaling or rotation formulas, particularly when dealing with fixed points or pivot points.
3. **Incorrect Redrawing:** Forgetting to clear the screen between frames can lead to visual clutter, where multiple polygons overlap without proper differentiation between the original and transformed shapes.
4. **Handling User Input:** Managing the input of vertices and parameters for transformations (like angles for rotation or factors for scaling) can be tricky. Input validation is essential to avoid errors such as applying invalid transformations.
5. **Data Structure Management:** The program uses two vectors to store the original and transformed vertices. Beginners may struggle with updating the transformed vector correctly, especially when applying multiple transformations sequentially.
6. **Transformation Order:** Applying transformations in the wrong order can lead to unexpected results. For example, performing translation before scaling will give a different outcome than scaling first, then translating.

Despite these challenges, successfully implementing the transformations provides valuable insight into how geometric operations are handled in computer graphics and reinforces understanding of linear algebra concepts like matrix transformations.

Conclusion:

This lab demonstrates how fundamental 2D geometric transformations can be applied to polygons in computer graphics using OpenGL. By applying translation, scaling, rotation, reflection, and shearing, the user can explore how geometric figures are manipulated within a 2D coordinate system. The successful implementation of these transformations provides a solid foundation for more advanced graphics operations, such as 3D transformations and animations.

Sure! Here's the **Algorithm** section written in the style of the sample you provided earlier:

Algorithm:

Step 1: Input the number of vertices and their coordinates

- **Input:**
 - Get the number of vertices (n).
 - Enter the coordinates (x_i, y_i) for each vertex (i), where $(1 \leq i \leq n)$.
 - Store the vertices in the `original` vector.

Step 2: Display the original polygon

- **Display:**
 - Plot the polygon using the vertices in the `original` vector.
 - Each vertex is plotted using OpenGL's `glVertex2f()` function.

Step 3: Provide a transformation menu

- **Transformation Menu:**
 - Display the following transformation options:
 1. Translation
 2. Scaling (Fixed Point)
 3. Rotation (Pivot Point)
 4. Reflection
 5. Shearing
 6. Quit

Step 4: Perform transformations based on user input

Case 1: Translation

- **Input:** Get translation factors (t_x) and (t_y).
- **Computation:**
 - For each vertex ((x_i, y_i)) in the original polygon: $[x'_i = x_i + t_x] [y'_i = y_i + t_y]$
 - Store the new vertices in the `transformed` vector.
- **Display:** Plot the transformed polygon.

Case 2: Scaling (Fixed Point)

- **Input:** Get scaling factors (s_x), (s_y), and fixed point (f_x), (f_y).
- **Computation:**
 - For each vertex ((x_i, y_i)) in the original polygon: $[x'_i = (x_i - f_x) \cdot s_x + f_x] [y'_i = (y_i - f_y) \cdot s_y + f_y]$
 - Store the new vertices in the `transformed` vector.
- **Display:** Plot the transformed polygon.

Case 3: Rotation (Pivot Point)

- **Input:** Get the rotation angle (θ) and the pivot point (p_x, p_y).
- **Computation:**
 - Convert the angle to radians: $\theta_{\text{radians}} = \theta \times \frac{\pi}{180}$
 - For each vertex ((x_i, y_i)) in the original polygon: $[x'_i = (x_i - p_x) \cdot \cos(\theta_{\text{radians}}) - (y_i - p_y) \cdot \sin(\theta_{\text{radians}}) + p_x] [y'_i = (x_i - p_x) \cdot \sin(\theta_{\text{radians}}) + (y_i - p_y) \cdot \cos(\theta_{\text{radians}}) + p_y]$
 - Store the new vertices in the `transformed` vector.
- **Display:** Plot the transformed polygon.

Case 4: Reflection

- **Input:** Choose an axis for reflection:
 - (a) -> X-axis
 - (b) -> Y-axis
 - (c) -> Origin
 - (d) -> Line ($x = y$)
- **Computation:**
 - For each vertex ((x_i, y_i)) in the original polygon:
 - If reflecting across the X-axis: $[x'_i = x_i, y'_i = -y_i]$
 - If reflecting across the Y-axis: $[x'_i = -x_i, y'_i = y_i]$
 - If reflecting across the Origin: $[x'_i = -x_i, y'_i = -y_i]$
 - If reflecting across the line ($x = y$): $[x'_i = y_i, y'_i = x_i]$
 - Store the new vertices in the `transformed` vector.
- **Display:** Plot the transformed polygon.

Case 5: Shearing

- **Input:** Get the shearing factor (sh) and choose the axis (X or Y).
- **Computation:**
 - For each vertex (x_i, y_i) in the original polygon:
 - If shearing along the X-axis: $x'_i = x_i + sh \cdot y_i, \quad y'_i = y_i$
 - If shearing along the Y-axis: $x'_i = x_i, \quad y'_i = y_i + sh \cdot x_i$
 - Store the new vertices in the `transformed` vector.
- **Display:** Plot the transformed polygon.

Step 5: Display the original and transformed polygons

- **Display:**
 - Render the original polygon in red and the transformed polygon in green to visually compare the transformation.

Step 6: Loop the process or exit

- **Repeat:**
 - Allow the user to choose and apply additional transformations or exit the program.

This version follows the style you provided, maintaining clarity and breaking down the steps as pseudocode. Let me know if this works for you!